

臺北市立大學資訊科學系

資訊專題報告

專題題目：Road Object Detection in Fisheye Camera

學號:U11016026

姓名:鄭瑞良

指導教授: 蔡俊明

中華民國 115 年 7 月

臺北市立大學資訊科學系

大學部資訊專題報告審查表

學號	U11016026	姓名	鄭瑞良
題目	Road Object Detection in Fisheye camera		
指導教授	蔡俊明		

評語：

It is great!

審查結果：☑通過　□修正後通過

□修正後再審　□不通過

日期：中華民國 115 年 7 月 10 日

＊修正、審查通過後，請務必重新繳交新報告文件(含.doc、.ppt或其他作品)給系辦及指導教授。

指導教授簽章：教授蔡俊明

臺北市立大學資訊科學系專題著作授權書

本授權書所授權之專題報告為授權人在臺北市立大學資訊科學

系所 115 級 114 學年度第 2 學期完成之資訊專題。

題目：Road Object Detection in Fisheye Camera

指導教授：蔡俊明

茲同意將本人擁有著作權之上列報告全文(含其他媒體資料)，以非專屬及無償方式授權本系將作品及報告放置於網站、提供借閱，或以紙本型式出版，且不限時間與地域，作為非營利學術研究目的之利用。

授權人姓名：鄭瑞良 (請親筆正楷簽名) 學號：U11016026

中華民國 115 年 7 月 10 日

ABSTRACT

Object detection is a cornerstone of modern computer vision applications, with the YOLO (You Only Look Once) family of models setting benchmarks for real-time performance and accuracy. Fisheye cameras, widely deployed in traffic surveillance systems, introduce significant geometric distortion that poses unique challenges for conventional object detection models. This study presents a comprehensive evaluation of nine YOLO model variants — YOLOv9c, YOLOv9e, YOLOv9-SE, YOLOv10l, YOLOv10x, YOLO11l, YOLO11x, YOLO12l, and YOLO12x — on the FishEye8K benchmark dataset, with the primary objective of identifying the most effective and efficient model for fisheye-based traffic surveillance. All models were trained under identical experimental conditions for 100 epochs using an input resolution of 640×640. Performance was evaluated across multiple metrics including mAP@0.5, mAP@0.5:0.95, Precision, Recall, F1-Score, and inference speed. Additionally, YOLOv9-SE, an architectural modification of YOLOv9 incorporating Squeeze-and-Excitation attention mechanisms, was evaluated to assess the impact of attention-based feature recalibration on detection performance. Experimental results demonstrate that YOLO11x achieves the highest detection accuracy with a mAP@0.5 of 0.329, while YOLOv9-SE attains the highest F1-Score of 0.381, indicating the most balanced trade-off between precision and recall among all evaluated models. These findings provide practical insights for selecting and optimizing YOLO-based architectures for deployment in fisheye traffic surveillance environments.

TABLE OF CONTENTS

1. Introduction **2**

2. Related Works **3**

2.1. FishEye Camera Datasets 3

2.2. Test Datasets 5

2.3 Detection Model for Training 6

3. Training Procedure **10**

3.1 Metrics 10

3.2 Results 11

4. Discussions **17**

4.1 Increasing epoch when training 17

4.2 Fine-Tuning with Mosaic Augmentation 18

4.3 Modifying Confidence and IoU Thresholds 20

5.0 Conclusion **21**

Reference **21**

1. Introduction

Road object detection is essential for applications like autonomous driving, intelligent transportation systems, and traffic management. Accurately identifying road objects—such as vehicles, pedestrians, cyclists, and traffic signs—ensures safety and efficient navigation. For example, in autonomous vehicles, real-time object detection allows the system to make quick, informed decisions, enhancing safety and reducing accident risks. Traffic management systems also benefit from object detection, as they can monitor road usage, manage congestion, and optimize traffic flow, thus contributing to smarter, more responsive cities.
The YOLO (You Only Look Once) series has been a major contributor to advancements in real-time object detection, known for balancing speed and accuracy. Notable versions, such as YOLOv9[2], YOLOv10[3],YOLOv11[4] and YOLOv12[5] have introduced innovative improvements in accuracy and efficiency [1]

- **YOLOv9**: Created by Ultralytics, YOLOv9 features a streamlined architecture that enhances detection accuracy while supporting real-time processing, making it popular among researchers for its ease of use and comprehensive documentation [2].
- **YOLOv10**: Building on YOLOv9, YOLOv10 introduces further architectural optimizations and advanced training methods, achieving improved accuracy and speed. These enhancements make YOLOv10 especially effective for real-world applications, including dynamic road environments [3].
- **YOLOv11**: introduces significant improvements in architecture and training methods, making it a versatile choice for a wide range of computer vision tasks. [4].
- **YOLOv12** : introduces an attention-centric architecture that departs from the traditional CNN-based approaches used in previous YOLO models, yet retains the real-time inference speed essential for many applications. This model achieves high object detection accuracy through novel methodological innovations in attention mechanisms and overall network architecture, while maintaining real-time performance. [5]

Objective

This research aims to compare the performance of YOLOv9[2], YOLOv10[3], YOLOv11[4], and YOLOv12[5] for detecting road objects using the Fisheye8K [6] dataset. Key performance metrics—such as accuracy, speed, and handling of fisheye image distortion—will be evaluated to determine which model is best suited for real-time road object detection. This study seeks to inform future advancements in autonomous driving and traffic management by identifying the optimal YOLO version for these applications.

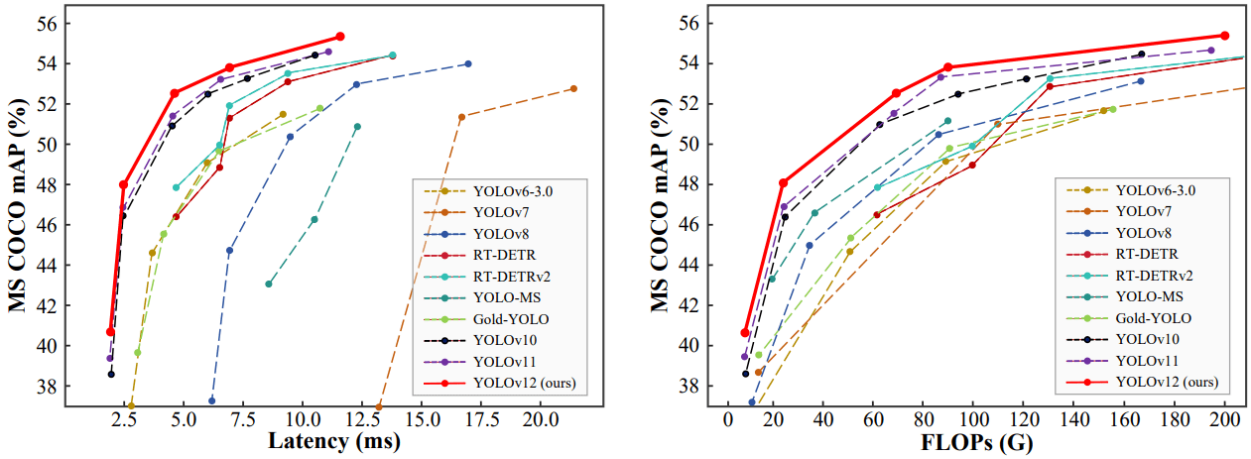


Figure 1. Comparison between the newest YOLO version to older versions [5]

2. Related Works

In this section, we present an overview of the landscape involving fisheye images and their related datasets, methodologies for object detection, ensemble techniques, and the most recent approaches for detecting small objects.

2.1. FishEye Camera Datasets

Fisheye images introduce specific challenges for object detection due to their unique distortion characteristics. A fisheye lens creates a hemispherical, wide-angle view that often covers more than 180 degrees, resulting in significant curvilinear distortion. This distortion warps object shapes, especially near the edges of the image, making it difficult for object detection models to recognize objects accurately. For instance, vehicles and pedestrians near the image boundaries may appear elongated or bent, potentially leading to misidentification by standard detection algorithms [6].

Additionally, fisheye images capture a broad field of view, introducing scale variability. Objects closer to the camera appear larger, while distant objects become much smaller. This range in object sizes presents a further challenge for detection models, especially for small objects like pedestrians or cyclists, which may be missed or inaccurately classified [6] .

A further complication is the relative scarcity of fisheye-specific datasets compared to standard datasets, limiting opportunities for training and fine-tuning models to handle these distortions effectively. Consequently, models trained on conventional datasets may struggle with accuracy when applied to fisheye images [6].

To address these challenges, prominent fisheye datasets such as FishEye8K provide large, annotated collections of fisheye images. The FishEye8K dataset, which serves as the basis for this research, consists of 5,288 annotated images with approximately 112,205 bounding box instances in the training dataset alone, making it one of the most comprehensive datasets available for fisheye-specific object detection [6].

These challenges underscore the importance of selecting or designing object detection models that are robust to the unique characteristics of fisheye images, particularly for applications like road object detection, where precise identification is essential for safety and efficiency [7].

Table 1. The distribution of instances across various classes within the FishEye8K dataset.

ID	Class	Number of Instances
0	Bus	2,054
1	Bike	62,261
2	Car	36,628
3	Pedestrian	9,126

4	Truck	2,136

five categories: ‘buses’, ‘bikes’, ‘cars’, ‘person’, and ‘trucks’

The distribution of instances across all classes is shown in Table 1, while sample images from the dataset are presented in Figure 2.

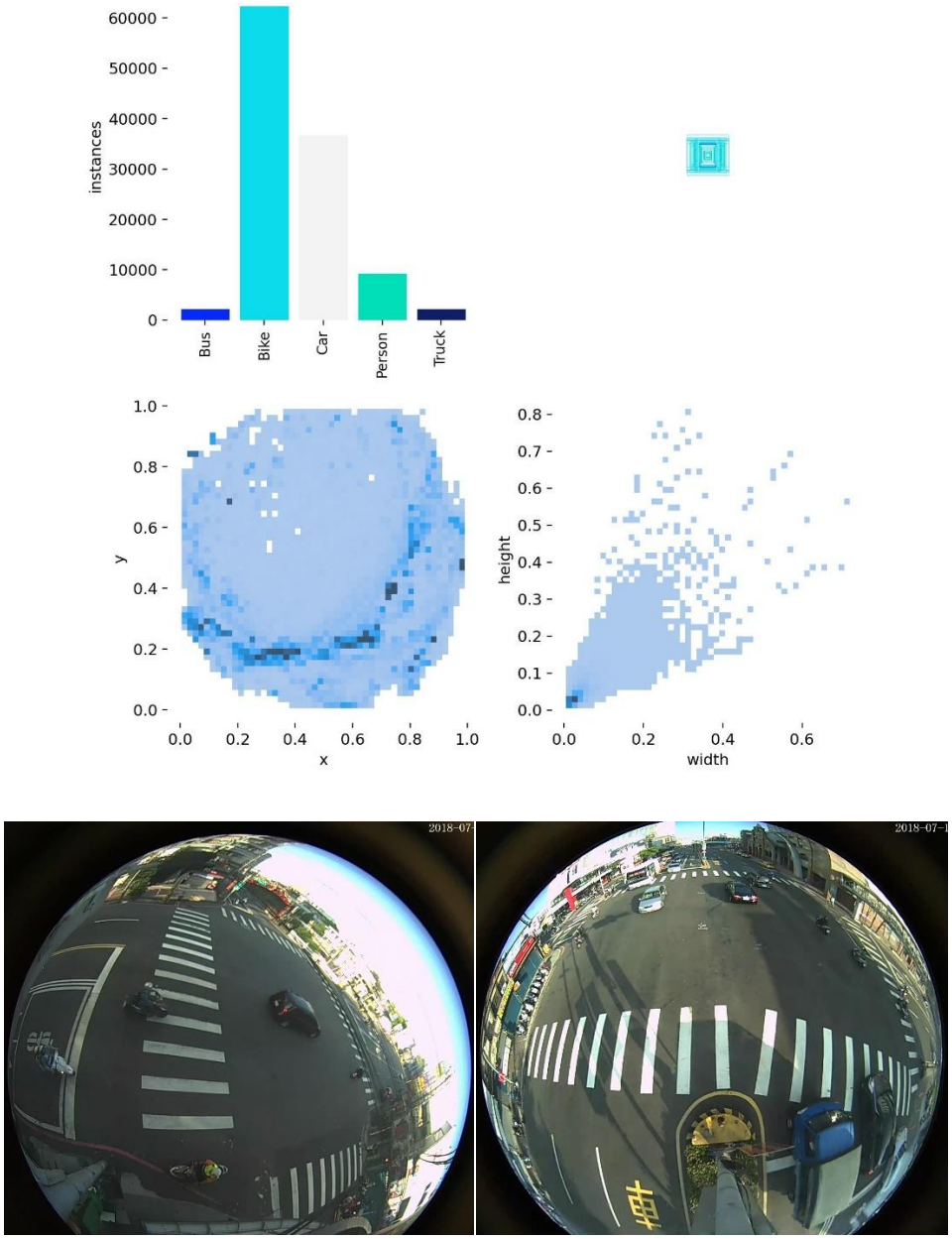


Figure 2. Fisheye8K train dataset samples [6]

2.2. Test Datasets

The trained model will be evaluated using a dataset of 1,000 images comprising 36,609 instances, entirely distinct from the images utilized during training.

Table 2. The distribution of instances across various classes within the test FishEye8K dataset.

ID	Class	Number of Instances
0	Bus	557
1	Bike	13,354
2	Car	15,945
3	Pedestrian	6,184
4	Truck	569



Figure 3. Original image (left) and predicted image using the trained YOLO11x model (right) [4]

2.3 Detection Model for Training

YOLO as a Modern Object Detection Framework

YOLO[8] (You Only Look Once) has emerged as a leading object detection framework due to its exceptional balance between speed and accuracy. It uses a single neural network to predict bounding boxes and object classes in a single forward pass, making it highly suitable for real-time applications like autonomous driving and traffic management. The latest YOLO[8] models continue to build on this foundation, introducing architectural and training optimizations to improve performance across diverse datasets and challenging environments.

In this study, we evaluate nine variants across four YOLO generations to determine their suitability for detecting road objects in fisheye images, focusing on metrics such as precision, recall, and inference speed.

The models evaluated in this study are YOLOv9[2]**, YOLOv10**[3]**, YOLOv11**[4]**, and YOLOv12**[5] **representing the latest advancements in the YOLO series for real-time object detection.**

YOLOv9:

YOLOv9 [2] addresses critical challenges in deep learning, such as information loss, by introducing innovative techniques like Programmable Gradient Information (PGI) and the Generalized Efficient Layer Aggregation Network (GELAN). These enhancements improve the model's feature extraction and gradient optimization, resulting in increased accuracy and speed. Its anchor-free detection head further streamlines object detection, enabling efficient real-time performance.

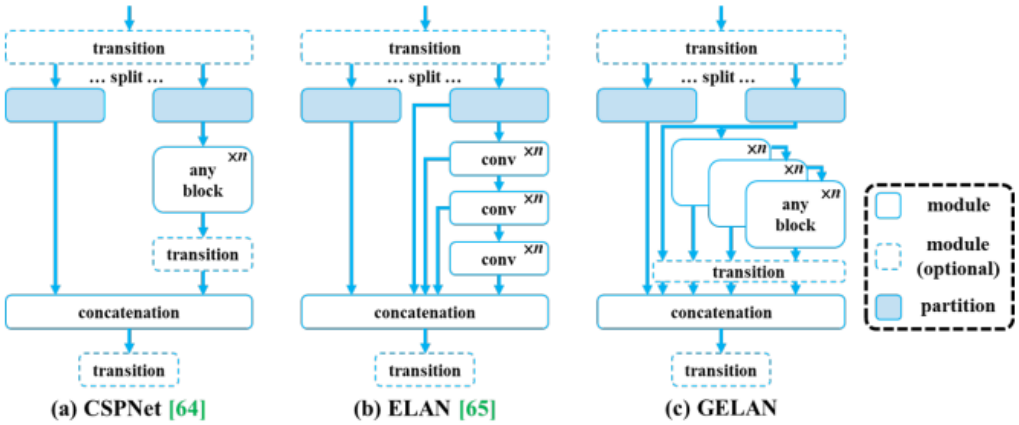


Figure 4. YOLOv9 architecture [2]

YOLOv9-SE:

YOLOv9-SE[11] using the YOLOv9[2] baseline model combine it with SE structure ,SENet is not an independent network model, but a module that can be combined with any existing model. The SE structure in SENet mainly consists of Squeeze, Excitation and Scale.

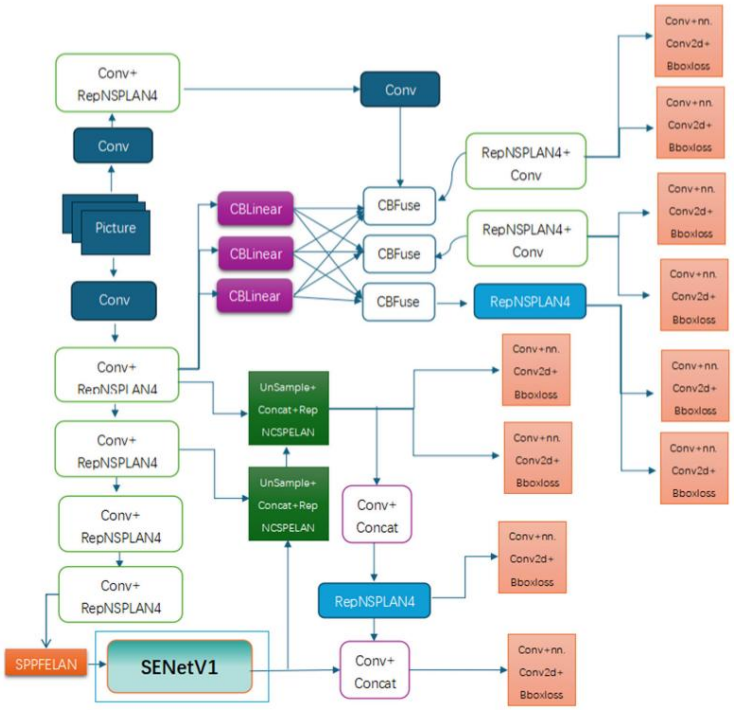


Figure 5. YOLOv9-SE architecture [11]

YOLOv10:

YOLOv10 [3] introduces a holistic efficiency-accuracy driven model design strategy that addresses key limitations of previous YOLO versions. Its most notable innovation is the consistent dual assignment strategy, which enables NMS-free end-to-end detection by eliminating the reliance on Non-Maximum Suppression (NMS) during inference. This reduces post-processing overhead and inference latency, making YOLOv10[3] particularly suitable for real-time deployment scenarios. Additionally, YOLOv10 incorporates spatial-channel decoupled down sampling and rank-guided block design, which collectively improve feature representation efficiency without substantially increasing computational cost.

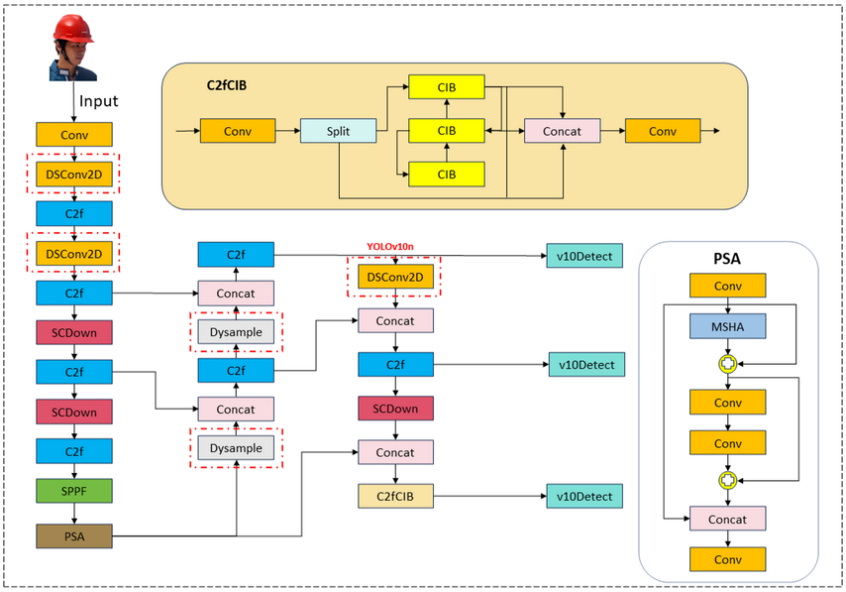


Figure 6. YOLOv10 architecture [3]

YOLOv11:

YOLO11 [4] is one of the latest additions to the YOLO series developed by Ultralytics, building upon the architectural foundations of YOLOv8. A key architectural innovation in YOLO11 is the introduction of the C3k2 block, which replaces the conventional C2f module and improves gradient flow during training, and the C2PSA (Cross Stage Partial with Position-Sensitive Attention) module, which enhances the model's ability to focus on spatially relevant features. These innovations contribute to improved detection accuracy across varying object scales, which is particularly beneficial for fisheye datasets where object sizes vary significantly due to lens distortion. YOLO11 is offered in five scaled variants from nano to extra-large, providing flexibility for deployment across diverse computational environments.

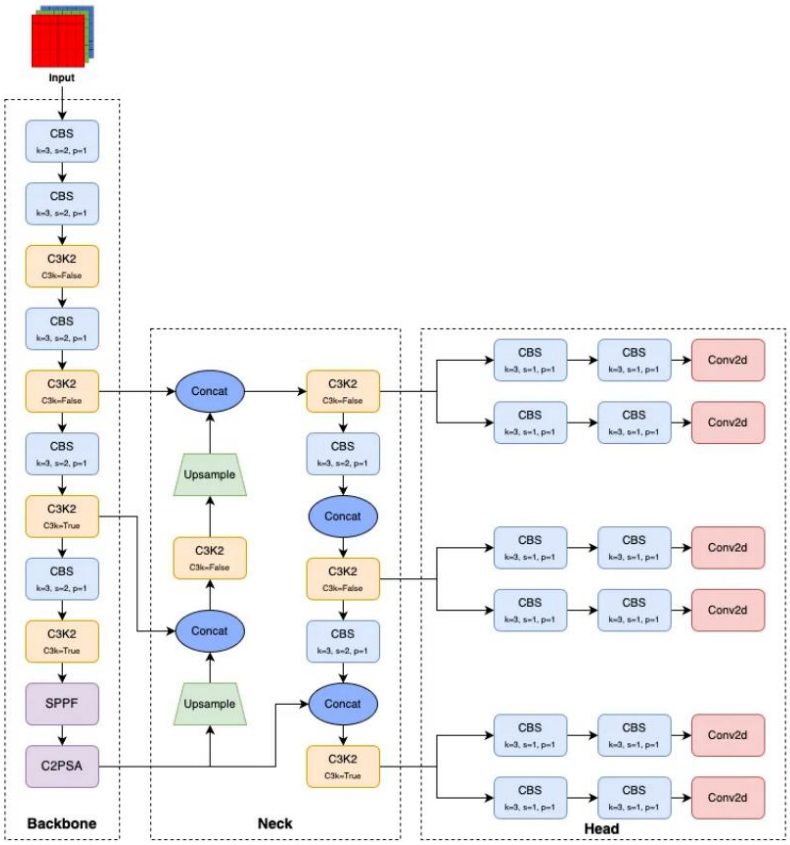


Figure 7. YOLO11 architecture showcasing the new C3k2 blocks and the C2PSA module [4]

YOLOv12:

YOLOv12 [5] represents a significant departure from the traditional CNN-centric design philosophy of previous YOLO models by introducing an attention-centric architecture. At its core, YOLOv12[5] employs an area attention mechanism that divides feature maps into local regions for efficient self-attention computation, significantly reducing the computational overhead typically associated with transformer-based architectures. This is complemented by a residual efficient layer aggregation network (R-ELAN) that improves feature aggregation stability during training. While YOLOv12[5] achieves competitive accuracy on standard benchmarks, its attention-based design may present challenges when generalizing to fisheye imagery, where significant geometric distortion alters the spatial distribution of features that attention mechanisms rely upon.

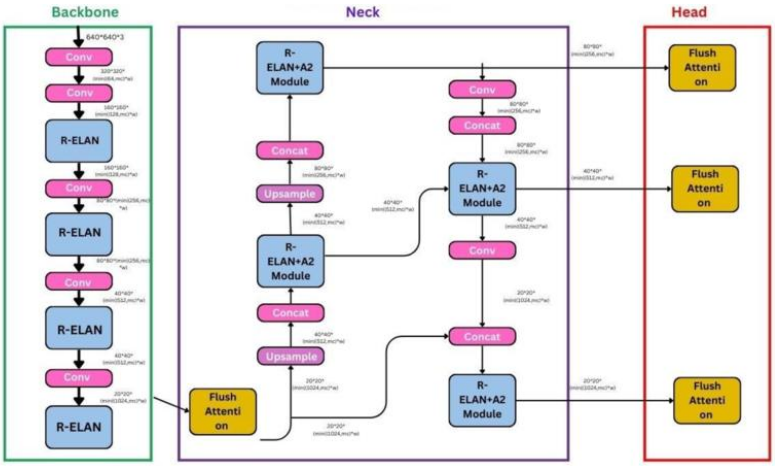


Figure 8. YOLOv12 architecture [5]

This study compares these models using the Fisheye8K dataset, evaluating their performance under real-world conditions, including distorted imagery and complex object scenarios.

3. Training Procedure

All experiments were conducted on Google Colaboratory Pro utilizing an **NVIDIA A100-SXM4-40GB** GPU with 12 logical CPU cores and a high-RAM runtime environment. All variations of YOLO were pretrained on the MS COCO dataset. Among these models, we trained eight variants: YOLOv9c [2], YOLOv9e [2], YOLOv9-SE [11], YOLOv10l [3], YOLOv10x [3], YOLO11l [4], YOLO11x [4], YOLO12l [5], and YOLO12x [5] using an input size of 640×640. All models underwent the same training procedures for 100 epochs with a batch size of 16, while all remaining hyperparameters were kept at their default values, including SGD optimizer with a momentum of 0.937, an initial learning rate of 0.01, and a weight decay of 0.0005. Data preprocessing. For training and testing, the input images were resized to 640×640.

3.1 Metrics

All models are evaluated using consistent metrics: Precision, Recall, mAPs F1-score, and inference time.

3.1.1 Precision

Precision measures the ratio of true positive detections to the total number of positive detections (including false positives). This metric reflects the model's ability to accurately classify vehicle classes without generating false positives. High precision indicates a lower rate of false detections, which is crucial in safety-critical applications**.**

$$Precision = \frac{True\ Positives}{True\ Positives + False\ Positives}$$

3.1.2 Recall

Recall calculates the ratio of true positive detections to the total number of actual positive instances (true positives and false negatives). A high recall score indicates that the model is effective at identifying most relevant instances, even in challenging situations with occluded or partially visible vehicles.

$$Recall = \frac{True\ Positives}{True\ Positives + False\ Negatives}$$

3.1.3 F1-Score

The **F1-score** metric reflects the balance between Precision and Recall. A high F1 score indicates strong model performance, as it suggests both Precision and Recall are high. Conversely, a low F1 score implies an imbalance between Precision and Recall, signaling suboptimal model performance. The F1 score is calculated as follows:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

3.1.4 Normalized Confusion Matrix

Normalized Confusion Matrix is used to determine the prediction quality of the model by each class. A confusion matrix is made up of 4 components, namely, True Positive (TP), True Negative (TN), False Positive (FP), and False Negative (FN).

3.1.5 Mean Average Precision

Mean Average Precision (mAPs) is the mean of the APs for all classes. The mAP of the object detection model is calculated according to:

$$mAP = \frac{1}{N}\sum_{k=1}^{n} AP$$

where n is the number of classes in the dataset and AP(k) is the average precision (AP) for a given class k.

- **mAP@0.5:** Average precision calculated at an IoU threshold of 0.5, primarily evaluating classification accuracy.
- **mAP@0.5:0.95:** Average precision calculated across IoU thresholds from 0.5 to 0.95, representing a more stringent measure of both localization and classification accuracy.

3.1.6 Inference Time

Inference time refers to the amount of time the model takes to process an input image and generate predictions. It is a critical metric for evaluating the real-time performance of object detection models. A lower inference time is crucial for applications requiring real-time performance, such as autonomous vehicles, surveillance systems, or robotics. However, achieving faster inference often involves trade-offs with accuracy.

3.2 Results

The primary goal of this study was to evaluate the performance trade-offs between inference speed and detection accuracy across multiple YOLO[1] model variants, including YOLOv9c[2],

YOLOv9e[2], YOLOv9-SE[11], YOLOv10l[3], YOLOv10x[3], YOLO11l[4], YOLO11x[4], YOLO12l[5], and YOLO12x[5], in order to determine their suitability for object detection tasks under fisheye camera conditions

3.2.1 Inference Speed

Inference speed, measured in milliseconds per image, is a critical factor in determining the real-time applicability of object detection models. Among all evaluated models, YOLO11l[4] achieved the fastest inference time at 4.4 ms, followed by YOLOv9c[2] at 4.8 ms, making both models highly suitable for latency-sensitive applications such as live video surveillance and real-time traffic monitoring

YOLOv9-SE[11] recorded an inference time of 7.2 ms, positioning it as a competitive mid-range option in terms of processing speed. YOLOv9-SE[11] represents an architectural modification of the YOLOv9[2] baseline, incorporating Squeeze-and-Excitation (SE) attention mechanisms into the network to enhance feature recalibration, with the primary objective of improving both detection accuracy and inference efficiency. The results indicate that this architectural modification successfully maintains a reasonable inference speed while achieving notable improvements in detection performance relative to the YOLOv9[2] baseline models, namely YOLOv9c[2] and YOLOv9e[2].

YOLOv9e[2] and YOLO12x[5] recorded the highest inference times among all models at 10.0 ms and 10.1 ms, respectively, suggesting that these models are better suited for offline or batch processing scenarios where computational latency is less of a concern.

3.2.2 Accuracy

In terms of detection accuracy measured by mAP@0.5, YOLO11x[4] achieved the highest score of 0.329, demonstrating superior object localization capability across the fisheye dataset. This was followed by YOLOv9-SE[11] at 0.320 and YOLOv9e[2] at 0.319, both of which exhibit competitive accuracy relative to their computational cost .

YOLOv9c[2] and **YOLOv9e**[2] both show competitive accuracy, with YOLOv9c[2] achieving an mAP@0.5 score of **0.314** and YOLOv9e[2] reaching **0.319**. While these values are slightly lower than those of YOLO11x[4], they still indicate a strong ability to perform object detection tasks reliably.

YOLOv9c[2] and YOLOv10x[3] recorded mAP@0.5 scores of 0.314 and 0.312, respectively, indicating reliable detection performance despite their relatively lower complexity. In contrast, YOLO12l and YOLO12x yielded the lowest mAP@0.5 values of 0.276 and 0.282, respectively, suggesting that the YOLOv12 architecture may not generalize as effectively to fisheye image characteristics without domain-specific fine-tuning.

3.2.3 mAP@0.5-0.95

The mAP@0.5:0.95 metric evaluates model performance across a range of Intersection over Union (IoU) thresholds from 0.5 to 0.95, providing a more comprehensive assessment of detection precision under varying levels of localization strictness.

YOLO11x achieved the highest mAP@0.5:0.95 score of 0.166, followed by YOLOv9e and YOLOv10x, both at 0.156, indicating robust localization performance across diverse detection thresholds. YOLOv9-SE recorded a score of 0.150, which, while slightly lower than the top-performing models, remains competitive given its architectural modifications and mid-range inference speed.

YOLO12l and YOLO12x recorded the lowest mAP@0.5:0.95 values of 0.132 and 0.137, respectively, further corroborating their relatively weaker performance under stricter localization criteria.

3.2.4 F1-Score

The F1-Score, which represents the harmonic mean of precision and recall, provides a balanced measure of a model's ability to minimize both false positives and false negatives simultaneously. Among all evaluated models, YOLOv9-SE achieved the highest F1-Score of 0.381, followed by YOLOv9c and YOLO11x, both at 0.373.

This result suggests that YOLOv9-SE offers the most balanced trade-off between precision and recall among all tested models, which is particularly significant in multi-class detection scenarios where both missed detections and false alarms carry practical consequences.

3.2.5 Trade-offs Between Speed and Accuracy

The experimental results reveal a nuanced relationship between inference speed and detection accuracy across the evaluated YOLO variants. Models such as YOLO11l and YOLOv9c prioritize computational efficiency, achieving inference times of 4.4 ms and 4.8 ms respectively, albeit at the cost of slightly reduced mAP scores. These models are well-suited for deployment in real-time systems with strict latency constraints.

Conversely, YOLO11x demonstrates that investing additional computational resources yields measurable gains in detection accuracy, achieving the highest mAP@0.5 of 0.329 at an inference time of 7.1 ms. This model is recommended for applications in which detection precision is paramount, such as autonomous vehicle perception or intelligent traffic management systems.

YOLOv9-SE, which represents an architectural modification of the YOLOv9 baseline through the integration of Squeeze-and-Excitation attention mechanisms, presents a compelling balance across all evaluated metrics. With the highest F1-Score (0.381), a competitive mAP@0.5 (0.320), and a moderate inference time (7.2 ms), the results demonstrate that this architectural enhancement yields consistent improvements in both accuracy and efficiency over the original YOLOv9 variants, without substantially increasing computational overhead. Among all models evaluated in this study, these findings position YOLOv9-SE as one of the most balanced candidates for fisheye-based traffic surveillance deployment.

Table 3. Results of state-of-the-art models trained on FishEye8K datasets.

Model	Input Size	Precision	Recall	mAP@0.5	mAP@0.5-0.95	F1-Score	Inference(ms)
YOLOv9-C [2]	640x640	0.486	0.303	0.314	0.154	0.373	4.8
YOLOv9-E [2]	640x640	0.455	0.313	0.319	0.156	0.371	10.0
YOLOv9-SE [11]	640x640	0.474	0.318	0.320	0.150	0.381	7.2
YOLOv10-L [3]	640x640	0.451	0.289	0.300	0.144	0.352	5.3
YOLOv10-X [3]	640x640	0.507	0.284	0.312	0.156	0.364	6.9
YOLOv11-L [4]	640x640	0.453	0.297	0.304	0.147	0.359	4.4
YOLOv11-X [4]	640x640	0.502	0.297	0.329	0.166	0.373	7.1
YOLOv12-L [5]	640x640	0.443	0.267	0.276	0.132	0.333	7.8
YOLOv12-X [5]	640x640	0.445	0.276	0.282	0.137	0.340	10.1

Figure 9 presents a visual comparison of all evaluated models across the six performance metrics.

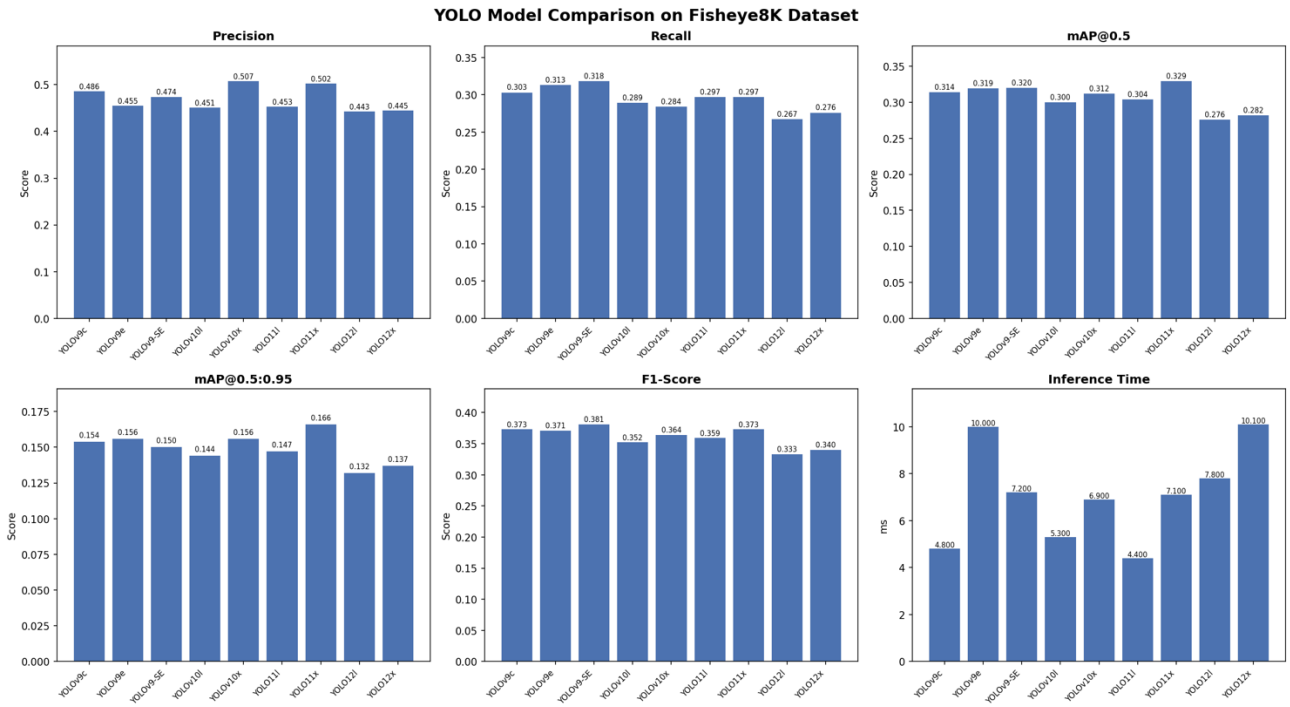


Figure 9. YOLO model comparison on FishEye8K dataset

YOLOv9-SE

The normalized confusion matrix of the YOLOv9-SE model is presented in Figure 10, providing a detailed view of per-class detection performance.

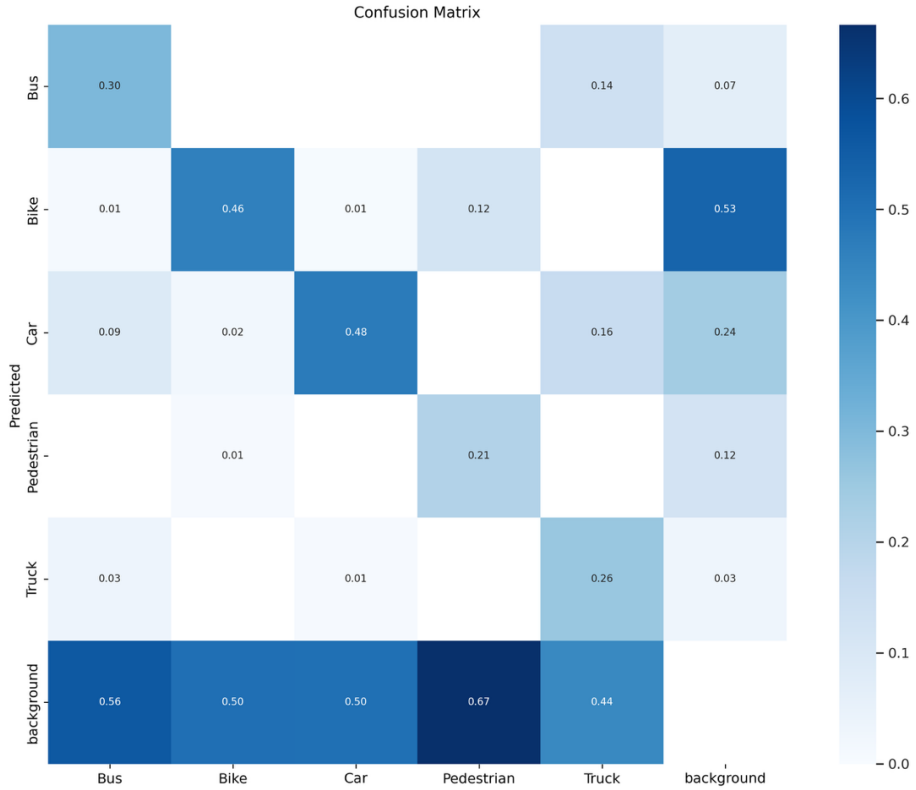


Figure 10. Normalized confusion matrix of YOLOv9-SE model on the input size 640 × 640

Table 4. Detail test result using YOLOv9-SE-640x640 model using val.py

YOLOv9-SE-640x640					
Classes	***Precision***	***Recall***	***mAP0.5***	***mAP.5-0.95***	***F1-Score***
Bus	0.226	0.307	0.207	0.126	0.260
Bike	0.534	0.405	0.402	0.160	0.461
Car	0.765	0.447	0.537	0.273	0.564
Person	0.553	0.180	0.252	0.090	0.272
Truck	0.293	0.251	0.204	0.100	0.270
All	0.474	0.318	0.320	0.150	0.381

In the YOLOv9-SE[11] model with a 640x640 input size, the Car class achieved the highest mAP@0.5 score of 0.537, alongside the highest F1-Score of 0.564 and Precision of 0.765, indicating that the model performs most reliably in detecting cars within the fisheye dataset. The Bike class followed with a mAP@0.5 score of 0.402 and an F1-Score of 0.461, demonstrating moderate detection capability for this class. In contrast, the Bus and Truck classes recorded

comparatively lower mAP@0.5 scores of 0.207 and 0.204, respectively, which can be attributed to the limited number of instances available for these classes during training. The Pedestrian class exhibited the lowest recall of 0.180, resulting in an F1-Score of 0.272, suggesting that the model frequently fails to detect pedestrians, likely due to their small apparent size under fisheye distortion and the inherent challenges of detecting deformable objects at varying scales.

YOLOv11-X

The normalized confusion matrix of the YOLO11x model is illustrated in Figure 11.

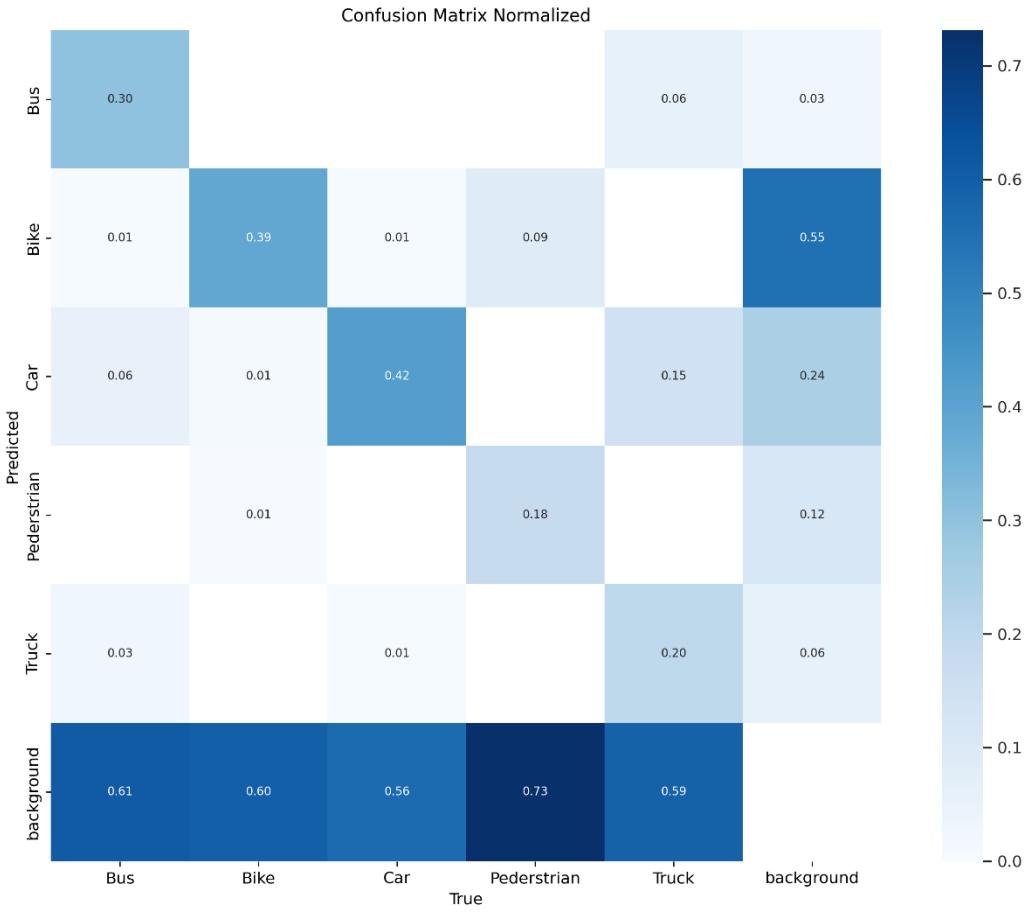


Figure 11. Normalized confusion matrix of YOLO11x model on the input size 640 × 640

Table 5. Detail test results using YOLO11x-640x640 model using val.py

YOLO11x-640x640					
Classes	***Precision***	***Recall***	***mAP0.5***	***mAP.5-0.95***	***F1-Score***
Bus	0.409	0.304	0.288	0.187	0.348
Bike	0.529	0.361	0.385	0.155	0.430
Car	0.783	0.414	0.542	0.292	0.543
Person	0.583	0.181	0.264	0.107	0.272
Truck	0.203	0.223	0.163	0.087	0.211

All	0.502	0.297	0.329	0.166	0.373

In the **YOLOv11x**[4] model with a 640x640 input size, the **Car** class achieved the highest mAP@0.5 score of **0.542**, indicating strong detection performance and high accuracy for this class. The **Bike** class followed closely, with a mAP@0.5 score of **0.385**, showing that the model is proficient in detecting bikes, although with slightly lower accuracy compared to the car detection.

However, the **Bus** class had the lowest mAP@0.5 score of **0.288**, suggesting challenges in accurately detecting buses. This indicates that the model might struggle with object classes that are smaller or less frequent in the training data. In terms of **Precision**, the **Car** class led the way, achieving **0.783**, reflecting that a significant portion of the car detections are correct. The **Bus** class, in contrast, showed the lowest precision of **0.409**, pointing to a higher rate of false positives.

Recall rates were also varied across classes. The **Car** class achieved the highest recall of **0.414**, showing that the model detects about 41% of the actual car instances. Meanwhile, the **Person** class lagged behind with a recall of **0.181**, indicating that the model struggles to detect people and misses a significant portion of actual instances. The **Bike** class achieved a decent recall of **0.361**, suggesting a moderate ability to detect bikes, though still below the **Car** class.

Overall, the YOLOv11x[4] model demonstrated **moderate detection capabilities** across various object classes with an average mAP@0.5 of **0.329** and a **mAP@0.5-0.95** of **0.166**, showing decent performance but with room for improvement, particularly for smaller or more challenging objects like buses and people.

4. Discussions

This study aims to enhance the accuracy observed in the results above by implementing additional strategies. For example, we plan to refine hyperparameter tuning, explore advanced augmentation techniques, or increase the training duration with optimized settings. These adjustments are intended to address potential shortcomings in the current model and maximize its performance

4.1 Increasing epoch when training

To further improve the accuracy and performance of our best model, YOLO11x[4], we extended the training duration from 100 to 200 epochs. Increasing the number of training epochs allows the model to learn additional patterns and features from the dataset, which can potentially enhance its ability to generalize to unseen data. This approach is aimed at refining the model's performance metrics, such as precision, recall, and mAP, while ensuring robust detection capabilities.

Table 6. Comparison between 100 epoch training and 200 epoch training

YOLO11x-640x640						
Epoch	***Precision***	***Recall***	***mAP0.5***	***mAP.5-0.95***	***F1-Score***	**Inference (ms)**

100	0.502	0.297	0.329	0.166	0.373	7.1
200	0.486	0.307	0.332	0.169	0.376	6.5

The results show a comparison of key performance metrics for YOLO11x[4] with input resolution 640x640 across 100 and 200 epochs. While the precision slightly decreased from 0.502 to 0.486, the recall increased from 0.297 to 0.307, leading to a marginal improvement in the F1-Score from 0.373 to 0.376. Similarly, the mAP@0.5 improved from 0.329 to 0.332, and mAP@0.5:0.95 increased from 0.166 to 0.169.

Interestingly, the inference time improved from 7.1 ms to 6.5 ms. This reduction might suggest slight computational efficiency gains as the model converges over more epochs. However, the minimal improvement in mAP and F1-Score suggests diminishing returns with extended training. This behavior indicates that the model might have reached its performance plateau, or the dataset may require additional preprocessing, hyperparameter tuning, or augmentation techniques to unlock further improvements.

4.2 Fine-Tuning with Mosaic Augmentation

To further optimize the YOLO11x[4] model, additional fine-tuning was performed using Mosaic[12] augmentation. This augmentation diversifies input data by combining multiple images during training, enhancing the model's ability to generalize.

Two fine-tuned models were developed:

1. **Model A**: Fine-tuned for an additional 50 epochs using the weights from a previously trained 100-epoch model.
2. **Model B**: Fine-tuned for an additional 50 epochs using the weights from a previously trained 200-epoch model.

In this setup, Mosaic[12] was applied throughout the training (`mosaic=1.0`, `close_mosaic=-1`), while other parameters such as the learning rate (`lr0=0.001`), batch size (`batch=16`), and optimizer (`SGD`) remained consistent. This approach allowed for a controlled evaluation of the impact of additional fine-tuning on model performance.

Table 7. Comparison of YOLO11x model with fine-tuning and mosaic augmentation

YOLO11x-640x640						
Epoch	***Precision***	***Recall***	***mAP0.5***	***mAP.5-0.95***	***F1-Score***	**Inference (ms)**
100	0.502	0.297	0.329	0.166	0.373	7.1
100+50 Fine-Tuning + Mosaic Augmentation	0.485	0.308	0.332	0.17	0.374	6.2
200	0.486	0.307	0.332	0.169	0.376	6.5
200+50 Fine-Tuning + Mosaic Augmentation	0.475	0.28	0.318	0.164	0.347	7.4

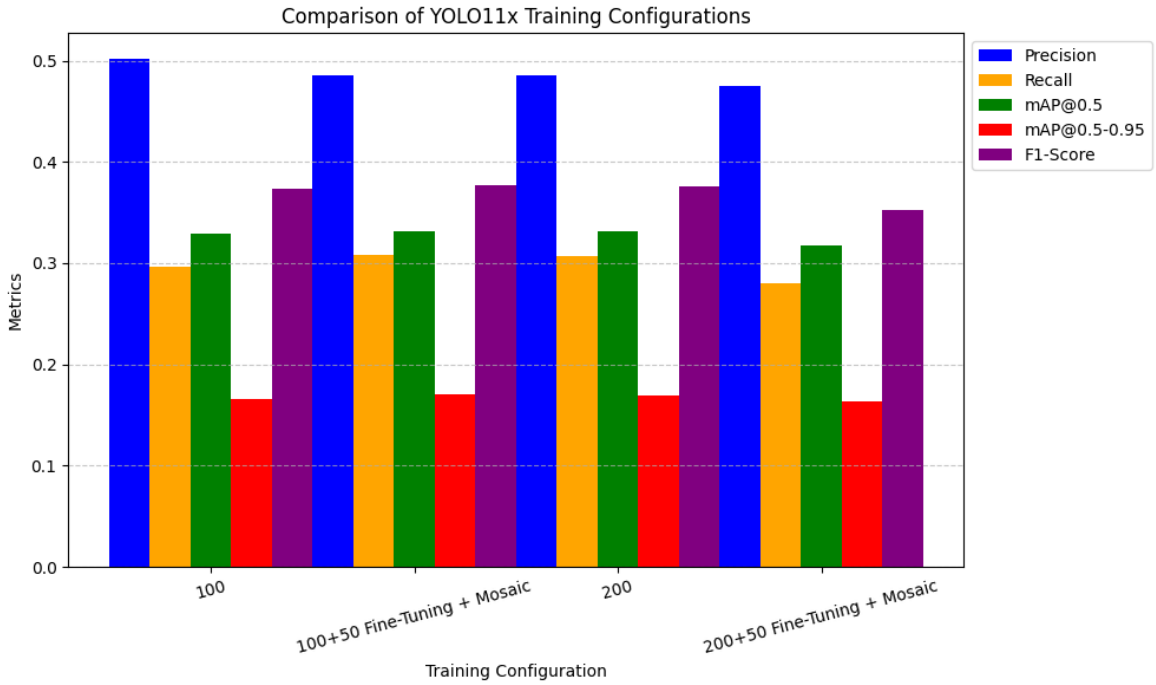


Figure 12. Comparison of YOLO11x training configurations

Key Observations

- Fine-tuning with Mosaic[12] augmentation after 100 epochs slightly improved recall and mAP@0.5-0.95 (from 0.166 to 0.170), with an F1-Score of 0.374. However, precision slightly decreased.
- Similarly, fine-tuning with Mosaic[12] augmentation after 200 epochs showed a trade-off, slightly reducing mAP@0.5 and precision while improving recall. The F1-Score decreased to 0.347.
- The inference time varied minimally across configurations, with the lowest being 6.2 ms for 100+50 Fine-Tuning + Mosaic and the highest being 7.4 ms for 200+50 Fine-Tuning + Mosaic.

The results indicate that Mosaic[12] augmentation combined with fine-tuning can modestly improve recall and mAP@0.5-0.95, potentially enhancing the model's detection capability in specific scenarios. However, these benefits come at the cost of a slight reduction in precision and F1-Score. This trade-off may be acceptable depending on the application's requirements, particularly if the focus is on recall rather than precision.



Figure 13. Original prediction (left) vs. fine-tuned prediction (right) using YOLO11x [4]

4.3 Modifying Confidence and IoU Thresholds

To improve the model's accuracy, we experimented with different configurations by modifying the confidence threshold (conf) and Intersection over Union (IoU) values during evaluation. By adjusting the confidence threshold, we aimed to fine-tune the balance between precision and recall, while changing the IoU allowed us to control the strictness of object detection. Specifically, we tested multiple settings for both parameters, including values of **conf = 0.25** and **IoU = 0.45**, and compared the results across different training epochs. Our findings indicated that adjusting the confidence and IoU thresholds led to variations in mAP scores, with different thresholds yielding slightly improved or worse performance for certain classes. However, after testing the model with 100 epochs versus 200 epochs, we concluded that the 200-epoch model did not consistently outperform the 100-epoch model, even with the modified settings. This suggests that, for our dataset, further fine-tuning beyond 100 epochs does not necessarily guarantee better accuracy.

Table 7. Performance comparison of YOLO11x under different confidence and IoU threshold settings (conf = 0.25, IoU = 0.45)

Test Model	*mAP0.5*	*mAP.5-0.95*
YOLOv11x 100 epoch	0.414	0.233
YOLOv11x 100 +50 Fine-Tuning + Mosaic Augmentation	0.419	0.238
YOLOv11x 200 epoch	0.392	0.22
YOLOv11x 200 +50 Fine-Tuning + Mosaic Augmentation	0.403	0.228

The performance of the YOLOv11x[4] model was tested across multiple configurations, focusing on the impact of training duration and augmentations. When comparing the results, it is evident that the model trained for 100 epochs with fine-tuning and Mosaic[12] augmentation

achieved the best overall performance, with a slight improvement in both mAP@0.5 (0.419) and mAP@0.5-0.95 (0.238). In contrast, extending the training to 200 epochs did not result in a better performance, with the mAP scores dropping slightly (mAP@0.5 = 0.392, mAP@0.5-0.95 = 0.22). This suggests that the combination of fine-tuning and augmentations may be more beneficial than simply increasing the number of training epochs, and that the 100-epoch model with these modifications yields superior accuracy.

5.0 Conclusion

In this study, we aimed to improve the performance of the YOLOv11x[4] model for object detection on a fisheye image dataset by experimenting with different strategies. We focused on fine-tuning the model, increasing the number of training epochs, and modifying the confidence and IoU thresholds during testing. These adjustments resulted in a notable improvement in the model's accuracy, with the mAP at 0.5 increasing from 0.329 to 0.419 and the mAP at 0.5-0.95 rising from 0.166 to 0.238.

Overall, YOLOv11x[4] demonstrated the highest accuracy among all models tested for this specific task. This suggests that YOLOv11x[4] is a strong candidate for object detection on fisheye images. Additionally, while smaller models might be better suited for real-time applications due to their faster inference speeds, YOLOv11x[4] offers a solid balance of accuracy, particularly when accuracy is prioritized over speed.

Reference

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition*, 2016, pp. 779–788.

[2] C.-Y. Wang, I.-H. Yeh, and H.-Y. M. Liao, "YOLOv9: Learning what you want to learn using programmable gradient information," arXiv preprint arXiv:2402.13616, 2024.

[3] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, and G. Ding, "YOLOv10: Real-time end-to-end object detection," arXiv preprint arXiv:2405.14458, 2024.

[4] G. Jocher and J. Qiu, "Ultralytics YOLO11," 2024. [Online]. Available: https://github.com/ultralytics/ultralytics

[5] Y. Tian, Q. Ye, and D. Doermann, "YOLOv12: Attention-centric real-time object detectors," arXiv preprint arXiv:2502.12524, 2025.

[6]M. Gochoo, A. Batnasan, M. Ganbayar, J. Junivay, and S. C. Huang, "FishEye8K: A Benchmark and Dataset for Fisheye Camera Object Detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR) Workshops*, 2023, pp. 5440-5447.

[7]N. J. Owor *et al.*, "A Unified Detection Pipeline for Robust Object Detection in Fisheye-Based Traffic Surveillance," 2024. [Online]. Available: https://arxiv.org/abs/2403.01234

[8] Z. Hua *et al.*, "A Benchmark Review of YOLO Algorithm Developments for Object Detection," *IEEE Access*, vol. 11, pp. 43011-43037, 2023.

[9] A. Chitari, "Deep Learning-Based NIR Face Detection under Adverse Illumination with Explainable AI," *J. Inf. Syst. Eng. Manag.*, vol. 10, no. 36s, pp. 613–625, 2025, doi: 10.52783/jisem.v10i36s.6541.

[10] L. Baoju *et al.*, "Safety helmet detection methods in heavy machinery factory," *Sci. Rep.*, vol. 15, 2025, doi: 10.1038/s41598-025-02824-z.

[11]Y. Gao, "YOLOv9-SE: A Real-time Vehicle Detection Model Based on Improved YOLOv9," *Trans. Comput. Sci. Intell. Syst. Res.*, vol. 8, pp. 137–145, 2024, doi: 10.62051/1zdpsr32.

[12] H. Zhang, S. Zhang, and R. Zou, "Select-Mosaic: Data Augmentation Method for Dense Small Object Scenes," *arXiv preprint arXiv:2406.05412*, Jun. 2024.